

OCEANSTUDIO — MASTER CODEX IMPLEMENTATION PROMPT

Repository:

<https://github.com/leonpresistforever-png/Oceanstudio-.git>

Product target for this phase: ANDROID APK FIRST.

You are the principal engineer responsible for turning OceanStudio into a real, production-capable, mobile-first AI coding, terminal, automation, research, creation and agent environment. This document consolidates the latest OceanStudio requirements into one authoritative implementation prompt. When two older requirements conflict, the newer Android-first instruction wins. Preserve unique features from earlier requirements, but do not carry forward obsolete Electron-first assumptions into the current APK implementation.

The product must be built around real execution. Never satisfy a requirement with a fake card, predefined list, hardcoded progress animation, fake provider, fake terminal, fake port, fake model response, fake MCP status, fake browser result, or simulated agent activity.

If Android, a provider, or an external service does not expose a required capability, represent the limitation truthfully and implement the closest legitimate architecture instead of fabricating success.

0. CURRENT PLATFORM PRIORITY

Build the Android APK first.

Do not spend this phase maintaining a separate Electron desktop implementation or website implementation. Keep reusable protocol, provider, MCP, storage and agent contracts portable where practical, but the Android app is authoritative.

Future desktop/web adapters may reuse:

- provider adapters
- tool schemas
- MCP definitions
- model metadata
- prompts
- protocol translators
- gateway logic
- storage schemas

Do not weaken the Android implementation merely to preserve theoretical cross-platform parity.

1. CORE ENGINEERING PRINCIPLES

1.1 No fake functionality

Do not create:

- fake terminals
- fake model availability
- fake provider connections
- fake execution logs
- fake progress
- fake ports
- fake file trees
- fake MCP servers
- predefined responses presented as network output
- static integrations presented as connected
- simulated tool calls
- fake auth states
- placeholder buttons presented as finished
- fake agent timers
- fabricated success states

Every visible operational state should come from the underlying system.

If a feature is incomplete, mark it incomplete.

1.2 Inspect before rewriting

Audit the existing repository before replacing architecture. Preserve real working systems. Repair broken real systems. Remove mock systems.

1.3 Verify external capabilities

For products and services such as OpenAI, Codex, Claude, Claude Code, Gemini, Google Jules, OpenRouter, Groq, Ollama, Cursor, Windsurf, Lovable, Firebase, Google Cloud, MCP servers and other integrations, determine which official interface actually exists:

- REST API
- SDK
- OAuth
- API key
- OpenAI-compatible endpoint
- Anthropic-compatible endpoint
- Gemini endpoint

- CLI
- MCP
- local HTTP API
- user-supplied endpoint

Do not invent undocumented account APIs.

1.4 Human aesthetic

OceanStudio must look designed by a strong product team, not by an AI dashboard generator.

Avoid:

- excessive gradients
- neon glow
- pulsing dots
- useless glass cards
- giant rounded rectangles everywhere
- cheap emoji icons
- random RGB outlines
- fake terminal decoration

Use calm typography, spacing, hierarchy and motion.

2. TECHNOLOGY ARCHITECTURE

Use languages because they are technically appropriate, not because they sound powerful.

Preferred architecture:

Android application and UI:

- Kotlin
- Jetpack Compose where compatible with the repository
- Android SDK

Agent/orchestration layer:

- Kotlin
- JetBrains Koog where it improves orchestration

Native system/terminal layer:

- C/C++
- Android NDK
- JNI

- AIDL/Binder where justified

Performance/security-sensitive portable modules:

- Rust where it materially improves safety or performance

Python:

- available inside Ocean Terminal, PRoot userlands, sandboxes and user projects
- usable for agent helper scripts and development environments
- do not embed a huge Python runtime into the APK core without a concrete requirement

Web technologies:

- valid for previews, generated projects, browser surfaces and certain playground outputs
- do not make the core Android application a web wrapper merely because HTML is easy

3. FIRST ACTION: REPOSITORY AUDIT

Before major implementation work, inspect the full repository and classify each substantial subsystem as:

- REAL + WORKING
- REAL + BROKEN
- PARTIAL
- MOCK/PLACEHOLDER
- DEAD CODE
- MISSING

Audit:

- Gradle/project structure
- Kotlin/Java
- native C/C++/Rust
- Firebase/auth
- terminal
- editor
- codebase explorer
- preview
- provider registry
- model selector
- streaming
- gateway/proxy
- agents
- Koog
- tool registry
- MCP

- extensions/plugins
- browser/research
- local models
- device automation
- Accessibility
- MediaProjection
- memory/database
- playgrounds
- tests
- security
- build pipeline

Search for indicators such as:

TODO, mock, placeholder, demo, fake, sample, hardcoded model lists, setTimeout-based execution, synthetic terminal output, static MCP success, fake provider state and fake browser results.

Do not rewrite real functionality merely because a different architecture would be more fashionable.

4. PRODUCT DESIGN LANGUAGE

OceanStudio should look like a premium modern AI/IDE product while retaining its own visual identity.

Design references include Claude, Gemini, Codex, Cursor, Linear and contemporary IDEs.

The supplied Figma references are design-language references. Use them to understand:

- spacing
- typography
- layout density
- information hierarchy
- sidebars
- conversational surfaces
- composer design
- transitions
- settings organization
- premium white-space

Do not clone proprietary branding or copyrighted assets. Derive a distinct OceanStudio design system.

Preferred visual language:

- white or very warm white background
- deep charcoal text
- restrained gray secondary text
- subtle shadows
- restrained borders
- calm accents
- generous spacing
- high-quality iconography
- meaningful micro-transitions

Figma reference URLs — preserve these in project documentation

Claude community reference:

https://www.figma.com/community/file/1645103147299035222/claude-redesign-concept-approachable-intelligence?q_id=ecac7ac3-1c7f-43b3-b937-7cc44e832141

Gemini community reference:

https://www.figma.com/community/file/1353986262501321287/google-gemini-ai-app-version?q_id=2e43131a-fb2c-4aef-843a-bac48065611a

Claude design file:

<https://www.figma.com/design/05FEDKH30gSBqsyKsJgzU5/Claude-redesign-concept---Approachable-Intelligence--Community->

Claude prototype:

<https://www.figma.com/proto/05FEDKH30gSBqsyKsJgzU5/Claude-redesign-concept---Approachable-Intelligence--Community-?node-id=0-1&t=yH3PFhUSIQDqb7nn-1>

Claude Dev Mode URL:

<https://www.figma.com/design/05FEDKH30gSBqsyKsJgzU5/Claude-redesign-concept---Approachable-Intelligence--Community-?node-id=0-1&m=dev&t=yH3PFhUSIQDqb7nn-1>

Claude embed URL:

<https://embed.figma.com/design/05FEDKH30gSBqsyKsJgzU5/Claude-redesign-concept---Approachable-Intelligence--Community-?node-id=0-1&embed-host=share>

Claude node URL:

<https://www.figma.com/design/05FEDKH30gSBqsyKsJgzU5/Claude-redesign-concept---Approachable-Intelligence--Community-?node-id=0-1&t=yH3PFhUSIQDqb7nn-1>

Shared Figma folder:

<https://www.figma.com/files/team/1668628294987296129/folder/637458238?fuid=1668628293613578318>

Gemini prototype:

<https://www.figma.com/proto/YAfT9eHsIs0zyl2P1kFVvU/Google-Gemini-AI-App-Version--Community-?node-id=10-129&t=azGi6T154TglMbnJ-1>

Gemini Dev Mode URL:

<https://www.figma.com/design/YAfT9eHsIs0zyl2P1kFVvU/Google-Gemini-AI-App-Version--Community-?node-id=10-129&m=dev&t=azGi6T154TglMbnJ-1>

Gemini embed URL:

<https://embed.figma.com/design/YAfT9eHsIs0zyl2P1kFVvU/Google-Gemini-AI-App-Version--Community-?node-id=10-129&embed-host=share>

Gemini node URL:

<https://www.figma.com/design/YAfT9eHsIs0zyl2P1kFVvU/Google-Gemini-AI-App-Version--Community-?node-id=10-129&t=azGi6T154TglMbnJ-1>

Use whichever URLs are accessible through the available Figma integration/browser. Do not block development if one community URL requires sign-in; use another supplied URL for the same reference.

5. AUTHENTICATION

Use the existing OceanStudio Firebase project/configuration already present in the repository.

Do not create new secret-bearing config files unnecessarily.

Authentication must actually work:

- email/password sign-up
- email/password sign-in
- Google sign-in
- forgot password/reset
- persisted session
- sign-out
- loading states
- clear errors
- retry
- disabled states during requests

Remove temporary login or auth bypass functionality.

The auth screen should remain extremely clean and premium and set the quality standard for the rest of the APK.

6. ONBOARDING / LAUNCH WORKSPACE

After authentication, provide a concise introduction and Launch Workspace action.

First-launch configuration may include:

- UI density/layout preference
- language
- agent approval behavior

Agent modes:

Review Driven:

- consequential operations require approval

Auto:

- agent acts independently for ordinary workspace operations
- asks for high-impact operations

Autonomous:

- reduced repeated confirmations for ordinary actions already within granted app permissions
- still cannot bypass Android security, permission prompts, service policy or user authentication

Persist these settings and allow changing them later.

7. MAIN WORKSPACE

Use a three-region architecture.

LEFT:

- Workspace
- Files
- Search
- Git
- Terminal
- Agents
- Models
- Providers

- MCP
- Extensions
- Automations
- Browser
- Playground
- Integrations
- Local Models
- Settings

The left sidebar must scroll and support clean expandable categories.

CENTER:

- Editor
- Codebase
- Preview
- Browser

RIGHT:

- agent/chat panel
- composer
- attachments
- tool activity

Do not put every configuration control on the main screen.

8. EDITOR AND CODEBASE

Implement a real mobile code editor.

Editor requirements:

- open real workspace files
- edit
- save
- create
- rename
- delete
- undo/redo
- syntax highlighting
- search
- language detection
- line numbers
- unsaved-state indication

- diagnostics where supported

Codebase explorer must reflect the actual filesystem.

Support recursive folders:

folder -> folder -> folder -> file

Operations:

- create file
- create folder
- rename
- move
- delete
- upload file
- upload folder through Android Storage Access Framework where possible
- refresh
- search
- export file
- export project ZIP

Never display sample folders as though they belong to the user.

Protect against unsafe path traversal outside approved workspace roots.

9. HARDWARE-LOCAL PREVIEW SYSTEM

Preview is not just a cosmetic WebView. Build a multi-backend Preview Router whose first preference is execution on the user's own hardware.

The user should be able to run a project from Ocean Terminal or an agent and see the actual result inside OceanStudio whenever Android technically permits it.

9.1 Local web preview

For HTTP/WebSocket/SSE servers started on-device:

- detect actual listening sockets/ports
- expose discovered localhost services to Preview
- load http://127.0.0.1:<port> or equivalent loopback address in an embedded browser/WebView
- support reload
- open full-screen
- select among multiple ports
- inspect URL

- terminate owning process when requested
- support WebSockets and streaming where WebView/network policy permits

Never invent a listening port.

Track ports from real process/session information and active socket checks.

9.2 Static/local project preview

For static HTML or generated assets:

- serve through a safe app-local HTTP server when browser security requires HTTP semantics
- avoid unsafe broad file:// access
- map project files to a sandboxed local origin

9.3 Linux GUI preview on the user's hardware

For GUI applications running inside Ocean Terminal or a PRoot distro, implement a fallback chain.

Preferred possibilities:

1. integrated on-device X server/rendering backend if a legally compatible maintained implementation is feasible;
2. on-device VNC server + embedded VNC client;
3. TigerVNC/Xvfb + noVNC/websockify loaded through OceanStudio's local Preview WebView;
4. external companion X server only as a fallback if an integrated implementation is not practical.

All of these should execute on the phone/tablet unless the user explicitly selects a remote runtime.

Do not claim arbitrary Linux GUI apps can render directly in Compose without an X11/VNC/Wayland-style display bridge.

9.4 PRoot GUI

Support a shared TMP/display strategy comparable to established Termux + PRoot X11 setups where appropriate. If using PRoot, make the display socket or VNC endpoint accessible to the preview backend in a controlled way.

9.5 Android APK preview

Android does not allow OceanStudio to simply embed another arbitrary installed application's Activity as though it were a child View.

For generated APKs:

- build the actual APK
- use PackageInstaller / normal Android installation flow with required user confirmation
- launch it through a valid intent
- optionally mirror its screen back into OceanStudio using user-authorized MediaProjection
- optionally enable interaction assistance only through user-enabled Accessibility APIs

Never bypass installation confirmation or secure-screen protections.

9.6 Native/CLI programs

Display CLI programs in Ocean Terminal. Programs that expose HTTP should flow into local web Preview. Programs that expose GUI through X/VNC should flow into the GUI preview backend.

9.7 Preview acceptance rule

A preview is real only if it originates from:

- a real local file/project
- a real running local service
- a real on-device GUI display endpoint
- a real installed/launched APK mirrored with user authorization
- or an explicitly selected remote runtime clearly labeled as remote

Never display a generated screenshot and call it a live preview.

10. RIGHT AGENT PANEL

Composer should provide:

- text entry
- model selector
- send
- stop
- voice where enabled
- attachments
- + menu

The + menu may include:

- Photo
- File
- Folder
- MCP

- Extension
- Integration
- Context

Attachments must actually become readable to the agent through controlled tool access.

11. AGENT ACTIVITY UI

Do not expose private chain-of-thought.

Expose concise operational activity instead:

Inspecting workspace

- files inspected
- summary

Running build

- actual command
- cwd
- stdout
- stderr
- exit code

Updating files

- filenames
- diff summary

Checking preview

- actual port/endpoint
- status
- validation

Show:

- real elapsed time
- current task
- completed operations
- tool calls
- logs
- errors
- retries

Expandable rows should expose useful logs without flooding the main chat.

No fake shimmering thinking text or timers detached from real activity.

12. OCEAN TERMINAL — PRODUCT IDENTITY

The built-in terminal subsystem is named:

OCEAN TERMINAL

Ocean Terminal is an OceanStudio subsystem, not a separate fake demo terminal.

Its goal is to provide a genuine Android-native command environment comparable in architecture to Termux while remaining OceanStudio-owned.

It must have:

- its own app-private filesystem layout
- its own home directory
- its own prefix
- its own bootstrap
- its own package manager UX
- its own package repository strategy
- its own terminal sessions
- its own environment variables
- its own process manager
- integration with Editor, Files, Agents and Preview

Do NOT use `/data/data/com.termux/...` as Ocean Terminal's runtime prefix.

Do NOT pretend ordinary unmodified Termux packages can simply be copied into another Android package namespace.

13. TERMUX RESEARCH BASIS FOR OCEAN TERMINAL

Before implementing Ocean Terminal's package/runtime architecture, study the current official Termux projects and preserve the architectural lessons, not the hardcoded package identity.

Official/reference sources:

Termux app:

<https://github.com/termux/termux-app>

Termux packages/build system:

<https://github.com/termux/termux-packages>

Termux file system layout:

<https://github.com/termux/termux-packages/wiki/Termux-file-system-layout>

Termux execution environment:

<https://github.com/termux/termux-packages/wiki/Termux-execution-environment>

Termux package management:

<https://github.com/termux/termux-packages/wiki/Package-Management>

Termux mirrors:

<https://github.com/termux/termux-packages/wiki/Mirrors>

Termux pkg wrapper implementation:

<https://github.com/termux/termux-tools/blob/master/scripts/pkg.in>

Termux PRoot-Distro:

<https://github.com/termux/proot-distro>

Termux:X11:

<https://github.com/termux/termux-x11>

Termux:GUI reference:

<https://github.com/termux/termux-gui>

Research conclusions that should directly influence Ocean Terminal:

1. Termux uses Android's private application data directory because normal Android apps cannot install arbitrary files into the system root filesystem.
2. Termux separates HOME from its package PREFIX.
3. Its prefix behaves as a virtual /usr and contains bin, etc, include, lib, libexec, opt, share, tmp and var.
4. Termux packages are built specifically for the expected Android execution environment and expected prefix. Hardcoded traditional Linux FHS paths are patched during package build.
5. Termux's `pkg` is a wrapper around a real underlying package manager such as apt or pacman; it is not a fake command parser.

6. Bootstraps contain a minimal package environment and are architecture/package-variant specific.

7. The Termux project explicitly advises forks not to use the official Termux package host as their package backend. OceanStudio therefore needs its own repository/mirror infrastructure if it distributes Ocean-specific packages.

8. PRoot-Distro demonstrates that rootless Linux userlands can run on Android without a Docker daemon or root, but they remain user-space environments with performance and kernel limitations.

9. Termux:X11 demonstrates a legitimate on-device X server approach, while VNC remains an important fallback.

Do not copy code without checking licenses, attribution, trademark implications and compatibility. Reuse open-source components only under their license obligations.

14. OCEAN TERMINAL FILESYSTEM

Derive the base path from Android APIs such as `applicationContext.filesDir`. Never assume only `/data/data/...`; Android may expose equivalent `/data/user/<id>/...` paths.

Conceptual layout:

```
$OCEAN_ROOT = <OceanStudio app-private files directory>/ocean-terminal
$OCEAN_HOME = $OCEAN_ROOT/home
$OCEAN_PREFIX = $OCEAN_ROOT/usr
$OCEAN_TMP = $OCEAN_PREFIX/tmp
$OCEAN_WORKSPACES = <OceanStudio controlled workspace root>
```

Prefix layout:

- \$OCEAN_PREFIX/bin
- \$OCEAN_PREFIX/etc
- \$OCEAN_PREFIX/include
- \$OCEAN_PREFIX/lib
- \$OCEAN_PREFIX/libexec
- \$OCEAN_PREFIX/opt
- \$OCEAN_PREFIX/share
- \$OCEAN_PREFIX/tmp
- \$OCEAN_PREFIX/var
- \$OCEAN_PREFIX/var/run

HOME must remain outside PREFIX so package reset/rebootstrap can replace package data without destroying user projects/configuration.

Environment variables should be derived dynamically, for example:

- OCEAN_ROOT
- OCEAN_PREFIX
- PREFIX=\$OCEAN_PREFIX for shell compatibility
- HOME=\$OCEAN_HOME
- PATH=\$OCEAN_PREFIX/bin:\$OCEAN_PREFIX/bin/applets:<approved system paths>
- TMPDIR=\$OCEAN_PREFIX/tmp
- TERM=xterm-256color
- COLORTERM=truecolor
- LANG/locale values supported by the runtime

Only set LD_LIBRARY_PATH or LD_PRELOAD when technically justified for a particular execution model. Do not globally inject unsafe shims into unrelated Android processes.

15. OCEAN TERMINAL PTY CORE

Do not present ProcessBuilder + text output as a Termux-equivalent terminal.

Use a real PTY architecture through NDK/JNI or a mature compatible library.

Required capabilities:

- master/slave PTY
- subprocess creation
- stdin/out/err routing through PTY as appropriate
- asynchronous read/write
- TIOCSWINSZ or equivalent resize
- process exit status
- signal delivery
- terminal session lifecycle
- environment/cwd handling

Ctrl+C must send the correct control input/signal behavior to the foreground process rather than clearing the UI.

Ctrl+Z, Tab, Esc, Alt and arrow keys must behave like terminal input.

16. TERMINAL RENDERER

Use a mature terminal-emulation core where possible.

Support:

- common ANSI sequences
- VT-compatible behavior
- xterm-256color where feasible
- cursor movement
- alternate screen
- scrollback
- Unicode
- selection
- clipboard
- resize
- pinch font scaling

Mobile extra-key bar:

- Ctrl
- Alt
- Tab
- Esc
- |
- ~
- -
- arrows

Allow user customization.

17. OCEAN BOOTSTRAP

Build architecture-specific Ocean Terminal bootstrap packages.

Prioritize arm64-v8a first.

Bootstrap should include enough to create a real package-managed environment, for example:

- shell
- core utilities
- TLS/certificate infrastructure
- curl/wget equivalent
- package manager and dependency tooling
- tar/compression
- Ocean pkg wrapper

First launch must:

1. verify bootstrap state/version/hash;
2. extract to a staging directory;
3. reject archive traversal (`.`, absolute escape paths, unsafe symlinks);
4. correctly distinguish directories from files;
5. apply executable modes;
6. atomically move staged prefix into place;
7. initialize HOME separately;
8. write repository/config metadata;
9. verify core commands;
10. recover safely from interrupted bootstrap installation.

Never fake extraction progress.

18. OCEAN PACKAGE MANAGER — `pkg`

Ocean Terminal must have its own `pkg` experience comparable to Termux.

The `pkg` command must be a real wrapper around the selected package-management backend, not a hardcoded command list.

Recommended first architecture:

- apt/dpkg-compatible Ocean package repository for Android-native Ocean packages
- Ocean `pkg` wrapper that simplifies install/search/update/upgrade/remove and repository health checks

Expected UX:

- `pkg update`
- `pkg upgrade`
- `pkg search <term>`
- `pkg show <package>`
- `pkg install <packages...>`
- `pkg remove <packages...>`
- `pkg list-installed`
- `pkg clean`
- `pkg change-repo` or equivalent repository selector

The underlying package manager must actually resolve dependencies and install real packages into \$OCEAN_PREFIX.

Do not point OceanStudio at the official Termux package host as the permanent package backend for a fork/product.

Build an Ocean package repository/mirror strategy.

Repository metadata must be signed and verified.

Package downloads should support:

- architecture metadata
- versioning
- dependencies
- checksums
- signatures
- resume/retry
- mirrors

Ocean package build recipes must patch or configure hardcoded FHS paths to target OCEAN_PREFIX where necessary, comparable to Termux's prefix-aware build process.

Do not promise “billions of Linux binaries.”

Classify executable environments:

- Android-native Ocean package
- shell script
- language package
- PRoot Linux package
- incompatible

19. OCEAN PACKAGE BUILD PIPELINE

Create a reproducible package-build system for Ocean Terminal.

It should support:

- package recipes
- source URL + checksum
- patches
- NDK cross-compilation
- ABI targeting
- prefix substitution
- dependency metadata
- package archive creation
- repository publication

- CI verification

Potentially study/fork compatible portions of termux-packages under license compliance, but Ocean-specific build recipes, repository URLs, prefix and signing must remain independent.

Do not compile packages directly on end-user devices when upstream build scripts are unsafe, destructive or too expensive. Prefer trusted reproducible CI builds for core packages.

20. PROOT LINUX ENVIRONMENTS

Provide optional rootless Linux userlands for software that is not practical as an Android-native Ocean package.

Support install/start/stop/remove/backup/restore for selected distributions such as:

- Alpine
- Debian
- Ubuntu

Potentially build on PRoot-Distro concepts or compatible tooling.

Make limitations clear:

- no real host root
- not a kernel VM
- syscall translation overhead
- Android process/background restrictions still apply

Allow controlled workspace mounts/binds and localhost network access.

Support Python, Node, compilers and server software inside the appropriate environment.

21. OCEAN TERMINAL + PREVIEW INTEGRATION

Every terminal session is registered with a process/session manager.

Track:

- PTY session ID
- PID/process tree where observable
- cwd
- environment type (native or PRoot)
- start time

- exit code
- active ports
- display endpoints

When a process starts a local HTTP service, Preview should discover and offer it.

When a Linux GUI service exposes X11/VNC/noVNC, Preview should offer the corresponding local display.

The agent must be able to request the same information through tools.

22. TERMINAL ACCEPTANCE TEST

Do not call Ocean Terminal real until these pass:

- interactive PTY shell launches
- `pwd` and `ls` execute actual binaries
- ANSI applications render correctly
- Ctrl+C interrupts a foreground process
- resize changes terminal geometry
- stdin/out streams incrementally
- process exit code is real
- HOME and PREFIX are app-private and separate
- bootstrap is integrity checked
- package manager resolves a real repository
- `pkg install` installs a real package
- newly installed binary executes
- package upgrade works
- local HTTP server becomes available to Preview
- PRoot environment can launch a supported distribution
- no output is fabricated

23. AGENT TOOL CONTRACT

Every capability must be represented by an explicit, schema-backed tool.

Core tool set:

1. `agent\_read\_file`

Inputs: path, optional line range

Outputs: content + metadata

2. `agent\_write\_file`

Modes: create, replace, patch

Return changed ranges/metadata

3. `agent\_grep\_search`

Inputs: query/regex, root, filters

4. `agent\_execute\_command`

Inputs: command, cwd, environment, session/timeout policy

Must route through the real process/Ocean Terminal layer.

5. `browser\_screenshot`

6. `browser\_navigate`

7. `browser\_click`

Browser tools may use WebView, local Chromium or an isolated PRoot browser backend.

Respect site security and policy. Do not implement CAPTCHA or anti-bot circumvention.

8. `sqlite\_vec\_query`

9. `android\_dump\_ui\_tree`

Requires user-enabled AccessibilityService.

10. `android\_inject\_gesture`

Uses documented dispatchGesture-style APIs after permission. Never claim it bypasses secure input protections.

11. `android\_launch\_intent`

12. `system\_notify`

Android notifications/haptics/device feedback through legitimate APIs.

13. `android\_ocr\_inspect`

OCR from user-authorized captured screen/image regions.

14. `process\_state\_diagnose`

Expose process/crash/lifecycle information legitimately available to the app and any explicitly configured privileged bridge.

15. `device\_policy\_manager`

Only expose DevicePolicyManager operations actually available under user-granted policy/admin conditions.

Tool result schemas must include errors rather than converting failure into fake success.

24. KOOG MULTI-AGENT ORCHESTRATION

Use JetBrains Koog where it improves structured orchestration.

Potential specialized workers:

- Planning Agent
- Coding Agent
- Debugging Agent
- Research Agent
- Refining/Reviewer Agent
- Task Execution Agent
- Vision/Screen Agent
- Automation Agent

Do not spawn all workers for every prompt.

An orchestrator decides which workers are needed.

Agents should exchange structured task artifacts rather than concatenated free-form messages.

25. WORKFLOW CHECKPOINTS

Persist long-running workflows.

Store:

- workflow ID
- active node
- task state
- completed tool calls
- relevant output
- file changes
- pending approvals
- timestamps

If Android kills the process, recover from the last safe checkpoint.

Use Room/SQLite and Android background primitives appropriately.

26. AGENT MEMORY

Implement controlled memory, potentially SQLite + sqlite-vec.

Separate:

- current conversation/session
- project memory
- semantic memory
- execution history
- preferences

Do not automatically vectorize secrets, credentials or every terminal log.

Provide:

- inspect memory
- delete entry
- clear project memory
- disable long-term memory
- retention settings

27. SELF-EXTENDING AGENT CAPABILITY

Agents may extend their controlled workspace by creating:

- AGENTS.md
- SKILL.md
- TOOL.md
- MCP.md
- scripts
- helper programs
- tool manifests
- workflow definitions
- provider adapters
- MCP configs
- local extensions
- requirements files

If a requested capability is missing:

1. inspect current tools;
2. identify missing capability;
3. search permitted registries/repositories;
4. propose/install dependencies within policy;
5. create adapter/tool;
6. test it;
7. register it;
8. use it.

The agent may modify its own workspace-level skills and helper code, but must not pretend it can mutate the installed APK binary in place. Native OceanStudio changes go through the repository/build/update process.

Do not allow self-extension to silently bypass Android permissions or plugin security.

28. AGENT DOCUMENTATION

Create detailed internal documentation for the main coding agent:

- repository layout
- workspace layout
- Ocean Terminal
- preview router
- file tools
- provider system
- gateway
- MCP
- browser
- device tools
- memory
- permissions
- debugging
- build commands
- testing
- how to add a tool
- how to add a provider
- how to add an MCP
- how to create a playground module

Keep playground-specific agent skills separate and focused instead of feeding the entire master prompt into every small tool.

29. PROVIDER REGISTRY

Remove fake providers and fake model selectors.

Build a provider registry with capability metadata.

Categories:

- API-key provider
- OAuth provider
- OpenAI-compatible endpoint
- Anthropic-compatible endpoint
- Gemini-style endpoint
- local model endpoint
- custom provider

Custom provider fields may include:

- name
- base URL
- auth type
- API key/token
- protocol
- headers
- models

Store secrets using Android Keystore-backed encryption.

30. MODEL GATEWAY / PROXY

Build an internal normalized gateway instead of scattering provider-specific logic through the UI.

Normalize internal objects for:

- messages
- instructions
- tools
- tool calls
- images
- files
- structured output
- usage
- errors
- streaming events

- supported reasoning metadata

Support OpenAI-compatible `/v1/\*` surfaces where useful for external IDE integration.

Do not assume every provider supports every field.

Maintain model capability metadata.

31. STREAMING

Responses must actually stream.

Implement:

- provider stream parsers
- incomplete-chunk buffering
- normalized deltas
- cancellation
- disconnect handling
- backpressure
- malformed-event handling
- retry policy

Do not expose private hidden chain-of-thought. If a provider offers user-facing reasoning summaries, treat them as explicit supported metadata rather than raw internal reasoning.

32. PROVIDER FALLBACK / COMBO ENGINE

Users can configure fallback chains.

Routing may consider:

- provider health
- recent success rate
- latency
- quota information when actually available
- rate limit
- cost
- model capabilities
- context length
- recent errors

Record the routing decision and reason.

Never invent quota remaining when an API does not expose it.

33. MULTI-PROVIDER COOPERATION

OceanStudio is the middle orchestrator.

Do not pretend one private provider directly gets another provider's private tools.

Use structured OceanStudio-owned task artifacts:

- task
- plan
- patch
- file reference
- tool request
- tool result
- review
- completion

Example:

Planning model -> OceanStudio task -> coding model -> Ocean tools -> reviewer model.

34. THIRD-PARTY IDE CONNECTIONS

Where documented, expose OceanStudio's compatible local gateway to external IDE clients.

Possible clients include Cursor, Cline, Zed or other configurable clients.

Do not claim account-level Cursor integration unless an official supported API exists.

Use documented base-URL/API mechanisms and protocol adapters.

35. OAUTH / PKCE

For OAuth-capable public mobile clients, use Authorization Code + PKCE with external/system browser where required.

Implement:

- code verifier
- challenge
- state verification
- redirect/App Link handling
- secure token exchange
- encrypted token storage
- refresh
- disconnect/revoke where supported

Do not embed reusable client secrets in the APK.

36. LOCAL MODELS

Create a real Local Models section.

Do not hardcode 100 fictional models.

Catalog real supported model metadata:

- purpose
- architecture
- size
- quantization
- RAM estimate
- storage requirement
- context length where known
- backend compatibility
- download state

Potential runtimes:

- llama.cpp
- ExecuTorch
- ONNX Runtime
- other Android-compatible engines

Support:

- download
- pause/cancel if backend supports it
- checksum verification
- delete
- load
- unload

- test inference

Classify models for coding, general assistant, vision, reasoning, embeddings and small/fast use cases.

37. MCP ARCHITECTURE

OceanStudio should operate as an MCP client and, where useful, a local MCP host/bridge.

Support only transports implemented by the selected MCP libraries/specification, such as:

- stdio
- Streamable HTTP
- supported SSE compatibility modes where needed

Each MCP server configuration should support:

- name
- transport
- executable
- arguments
- endpoint
- environment variables
- headers
- authentication
- working directory
- auto-start policy

Connection status must be real:

- Starting
- Connected
- Authentication required
- Failed
- Stopped

Do not mark a server connected merely because its config exists.

38. MCP CATALOG

Create a catalog of verified open-source MCP projects and templates.

Categories may include:

- Filesystem

- Git/GitHub
- Browser
- Database
- Python
- JavaScript
- Search
- Memory
- Blender
- Unity
- Sandbox
- DevOps

Each catalog item must show compatibility requirements:

- local Android native
- requires PRoot
- requires Node
- requires Python
- requires Docker/remote container
- desktop-only
- remote HTTP

Do not label something Android compatible without testing or verifying its runtime requirements.

Possible sandbox references requested for evaluation include:

- pottekkat/sandbox-mcp
- node-code-sandbox-mcp
- mcp-run-python
- chrishayuk/mcp-code-sandbox

Research current repositories, licenses and runtime requirements before integration.

Android cannot assume a Docker daemon is present. Use PRoot, WASM, restricted subprocesses or remote sandbox providers as appropriate.

39. EXTENSIONS / PLUGINS

Create a real extension architecture.

Extensions may contribute:

- tools
- agents
- playground modules

- commands
- model providers
- MCP configurations
- panels
- workflow templates

Manifest should declare:

- ID
- version
- entry point
- permissions
- dependencies
- compatibility
- exposed tools

Show requested permissions before installation.

Extensions should be isolated/sandboxed as much as practical.

Never display a static extension marketplace as though extensions are installed and working when they are not.

40. SANDBOX EXECUTION

Provide several backend classes rather than pretending Docker works everywhere.

Potential execution modes:

Restricted Process:

- Android app-owned subprocess with capability boundaries

PRoot:

- rootless Linux userland

WASM:

- Pyodide/WASI-compatible runtime for suitable tasks

Remote Sandbox:

- user-connected remote container/code execution provider

Each sandbox must clearly display:

- where code executes

- filesystem access
- network policy
- runtime limits
- persistence

41. BROWSER / RESEARCH SYSTEM

Build a real browser/research subsystem.

Possible browser backends:

- Android WebView for visible interactive browsing
- Chromium inside PRoot for local automation where stable
- Playwright/Puppeteer-compatible isolated browser where supported
- remote browser provider when explicitly selected

Capabilities may include:

- navigate
- search
- inspect DOM
- screenshot
- extract readable text
- click/type
- collect source metadata
- save page/article

Do not promise universal scraping.

Respect:

- authentication
- website restrictions
- CAPTCHA
- robots/policies where applicable
- provider terms

Do not implement anti-bot or CAPTCHA bypass mechanisms.

42. NEWS / READING / AUDIO BRIEFING

Playground tool: News Reader & Audio Briefing.

Capabilities:

- retrieve current articles from real feeds/search/providers
- open article
- save article
- summarize
- compare coverage
- import URL
- import PDF/document
- read full supported documents
- text-to-speech
- audio briefing
- playback speed
- voice selection
- queue
- storytelling/narration mode

Daily-news cards must come from real current sources rather than predefined JSON.

43. DEVICE INTEGRATION

OceanStudio may interact with the user's Android device only through legitimate user-authorized Android capabilities.

Potential components:

- AccessibilityService
- MediaProjection
- notifications
- haptics
- share intents
- Storage Access Framework
- camera/gallery
- microphone
- Android intents
- foreground services

Build a permission center that shows:

- permission/capability
- why it is requested
- enabled status
- affected features
- revoke/disable guidance

Never silently enable Accessibility or screen capture.

44. SCREEN UNDERSTANDING

When screen assistance is explicitly enabled:

Use structured Accessibility nodes first when available.

Use visual processing when needed:

- MediaProjection frame
- OCR
- OpenCV
- vision model

Do not run 60 FPS AI analysis continuously by default.

Use adaptive capture frequency based on task requirements and power/thermal state.

45. ACCESSIBILITY AUTOMATION

Accessibility-based tools may:

- inspect active nodes
- identify actionable controls
- tap
- swipe
- enter text
- navigate

only after the user enables the service.

Use documented Android APIs.

Never claim Accessibility gestures bypass secure UI, permission prompts or protected windows.

46. ASSISTIVE CODING AGENT

Users typing manually may enable a lightweight coding assistant.

Capabilities:

- diagnostics
- completion suggestions
- explain error
- quick fix
- refactor
- generate tests
- apply patch after approval according to chosen agent mode

Allow it to be disabled entirely.

47. PROJECT IMPORT / EXPORT

Support:

- Import file
- Import folder
- Export current file
- Export project ZIP
- Share project
- Duplicate workspace

Use Android Storage Access Framework correctly.

Do not expose arbitrary host paths outside the user-granted scope.

48. PLAYGROUND ARCHITECTURE

Playground tools are independent focused applications using shared OceanStudio infrastructure.

Every playground tool should have:

- dedicated screen
- actual backend capability
- model/provider selector where relevant
- files/images inputs where relevant
- processing status
- history
- export
- errors

Do not put all playground tools on the main workspace screen simultaneously.

49. PLAYGROUND — BANNER AD GENERATOR

Input:

- product description
- product URL
- brand assets
- logo
- optional references

Generate standard banner-ad sizes.

Capabilities:

- headline
- copy
- CTA
- image generation
- layout variants
- safe-zone checks
- export

Use real image generation/editing providers when configured.

50. PLAYGROUND — EMAIL CAMPAIGN STUDIO

From a prompt generate:

- subject lines
- preview text
- body copy
- CTA
- segmentation ideas
- visuals
- variants
- follow-up sequences

Allow tone/campaign objective controls.

Do not send mail without a legitimate connected email provider and explicit user authorization.

51. PLAYGROUND — VIRTUAL STYLIST

Inputs:

- clothing item photo
- optional full-body user reference
- wardrobe images

Analyze:

- palette
- garment type
- style
- season
- formality

Generate outfit concepts such as:

- Casual
- Business
- Night Out

Generate clean flat-lay images where the provider supports it.

Optional virtual try-on can use legitimate image-generation/editing APIs.

Pose/shot options may include:

- front
- rear
- side
- elevated
- low
- close
- full body
- user-described pose

Do not promise perfect garment physics or exact identity consistency beyond the selected model's documented capability.

52. PLAYGROUND — ROOM ORGANIZER

Input room photos.

Provide:

- clutter analysis
- categorization
- storage suggestions
- layout suggestions
- room zones
- actionable checklist
- optional before/after visualization

53. PLAYGROUND — LOCATION ASSISTANT

Use legitimate map/location services.

Capabilities may include:

- map display
- nearby search
- directions
- location-specific Q&A
- saved places
- route summaries

Google Maps integration must use official APIs/auth where selected.

Opening the installed Maps app should use valid Android intents.

54. PLAYGROUND — GARDENING ASSISTANT

Input:

- plant photo
- symptoms
- environmental details

Provide:

- identification candidates
- confidence
- watering
- light
- soil
- temperature
- care schedule

- common issues

Offer reminders through Android notifications after user permission.

55. PLAYGROUND — MERCH / PRODUCT MOCKUPS

Inputs:

- logo/product image
- optional brand guide

Generate visualizations for supported mediums such as:

- shirts
- mugs
- packaging
- billboards
- posters

Controls:

- background removal
- placement
- scale
- image quality
- variants
- export

Maintain source logo/product identity as much as the provider supports.

56. PLAYGROUND — SOCIAL CONTENT STUDIO

Input one idea and a tone such as:

- professional
- witty
- urgent
- custom

Generate platform-specific outputs:

LinkedIn:

- longer professional form

X/Twitter:

- concise/punchy version

Instagram:

- visual caption
- hashtags

Generate platform-appropriate images/aspect ratios where provider supports it.

57. SOCIAL SCHEDULER

Only publish through officially supported APIs/accounts.

Provide:

- drafts
- calendar
- account selection
- scheduling
- publish status
- retry/failure handling

Never show "published" unless the platform API confirmed success.

58. SOCIAL TREND RESEARCH

Use public search and supported social APIs.

Possible outputs:

- trending topic
- example sources/content
- publication date
- engagement metrics where legitimately available
- recurring format
- content style
- AI-generated opportunity ideas

Clearly separate observed data from model recommendations.

59. VIDEO CREATION WORKSPACE

When supported by connected video-generation providers:

- prompt
- reference assets
- shot list
- duration controls
- clip generation
- timeline
- regenerate scene
- change camera instruction
- continuity references
- combine clips
- export

Do not expose unsupported duration/camera controls as working.

60. PLAYGROUND — SOCRATIC MATH TUTOR

The math tutor should teach instead of immediately giving final answers.

Flow:

1. understand the problem;
2. guide the first meaningful step;
3. let the learner respond where appropriate;
4. explain "why" when asked;
5. adapt explanation depth;
6. continue incrementally.

Capabilities:

- algebra
- calculus
- geometry
- diagrams/simulations where helpful
- quizzes
- concept explanations
- homework generation
- progress tracking

61. VOICE / AUDIO INFRASTRUCTURE

Build shared voice components usable by playground tools:

- speech-to-text
- text-to-speech
- playback
- voice selection
- audio file input
- generated narration
- optional voice transformation where lawful/supported

Always indicate whether the engine is:

- local
- cloud
- unavailable

62. SECURITY ARCHITECTURE

OceanStudio is a powerful execution environment and must be designed defensively.

Secrets:

- API keys
- access tokens
- refresh tokens

must be encrypted using Android Keystore-backed storage.

Never print secrets in agent logs, diagnostic logs or terminal history unless the user intentionally typed them and the shell naturally contains them. Redact them from app telemetry.

Terminal/file subsystem:

- validate app-owned internal path operations
- defend bootstrap/archive extraction
- clearly label execution environment

Plugins/extensions:

- declared capabilities
- permission review

Device control:

- rely on user-granted Android permissions

Network:

- proper TLS validation

Database:

- no plaintext credential storage

63. TOOL PERMISSION MODEL

Each tool declares capability tags, for example:

- read_workspace
- write_workspace
- execute_process
- network
- browser
- install_package
- device_screen
- accessibility
- notification
- external_account

Review Driven mode asks for consequential permissions.

Auto asks mainly for high-impact operations.

Autonomous reduces repeated confirmations only for already-authorized ordinary operations.

No mode bypasses Android or service security.

64. DATABASE

Use proper structured persistence with migrations.

Potential entities:

- local profile
- workspaces
- provider definitions
- provider connections
- model metadata
- sessions
- messages
- tool calls

- workflow checkpoints
- memories
- MCP servers
- extensions
- playground histories
- usage telemetry
- preferences

Do not put all critical state into arbitrary JSON files.

65. OBSERVABILITY / DIAGNOSTICS

Build a developer diagnostics screen.

Expose real state:

- Ocean Terminal sessions
- running app-owned processes
- detected ports
- agent workflows
- model/provider calls
- provider latency
- tool calls
- MCP connections
- memory/database status
- preview endpoints
- errors/crashes

Allow export of sanitized diagnostics.

66. TESTING REQUIREMENTS

A feature is not complete because the UI renders.

Authentication tests:

- signup
- sign in
- Google sign in
- reset password
- session restore
- sign out

Ocean Terminal tests:

- PTY launches
- interactive command
- Ctrl+C
- resize
- bootstrap
- pkg repository
- package install
- installed binary executes
- stdout/stderr behavior
- exit code

Files/editor tests:

- create
- edit
- rename
- delete
- nested folders
- import
- export

Preview tests:

- run real local HTTP server
- detect actual port
- load it inside Preview
- stop server
- ensure preview reports stopped state

Agent test:

1. read file;
2. modify file;
3. run build/command;
4. inspect error;
5. fix;
6. rerun;
7. report exact change.

Provider test:

- with credentials, make real request
- without credentials, show setup state
- invalid credentials, show real auth failure

MCP test:

- initialize
- capability discovery
- list tools
- invoke a real tool
- disconnect/reconnect

Persistence test:

- recreate Activity/process where practical
- recover workspace/session state

67. HARDWARE PREVIEW ACCEPTANCE TEST

At minimum validate all applicable preview paths:

Web:

- Ocean Terminal starts a real local server
- Preview detects real port
- WebView loads it
- WebSocket/streaming works where required

Linux GUI:

- supported PRoot/native GUI app renders through integrated X/VNC/noVNC backend on the device

Android app:

- generated APK builds
- user approves install
- app launches
- optional MediaProjection mirror reflects the real launched app after explicit screen-capture consent

Do not call a preview path supported until its actual end-to-end test passes.

68. AGENT ACCEPTANCE TEST

Example request:

"Create a basic web server, run it and show me the preview."

The agent must actually:

1. inspect workspace;
2. create files;
3. install dependency if legitimately required;
4. launch the real server through Ocean Terminal/process tools;
5. detect the real listening port;
6. open Preview;
7. make a verification request;
8. report changes and endpoint.

Every operation displayed in activity UI must correspond to real execution.

69. PROVIDER ACCEPTANCE TEST

If disconnected:

- show Connect

If invalid credentials:

- show authentication failure

If connected:

- show real configured/discovered models

If the provider lacks a model-list endpoint:

- use a maintained verified manifest
- clearly identify that the list comes from maintained metadata rather than live enumeration

Never invent provider model availability.

70. MCP ACCEPTANCE TEST

Operational MCP requires successful:

- initialize
- capabilities
- tool discovery
- real invocation
- response validation
- reconnect
- error handling

Only then label it operational.

71. PERFORMANCE / THERMAL MANAGEMENT

Do not permanently run all heavy components together.

Do not continuously keep active without need:

- multiple sub-agents
- Chromium
- MediaProjection
- OCR
- vision model
- local LLM
- PRoot distro
- X server
- package updater

Use:

- lazy initialization
- process cleanup
- model unload
- lifecycle-aware workers
- streaming/backpressure
- memory monitoring
- cancellation
- battery-aware scheduling
- adaptive screen analysis

72. BACKGROUND EXECUTION

Use Android primitives properly:

- foreground services for user-visible ongoing execution that requires them
- WorkManager for deferrable background work
- coroutines
- lifecycle scopes
- Binder/AIDL where cross-process boundaries are justified

Do not use endless unmanaged threads as a persistence strategy.

73. OPEN-SOURCE / LICENSE MANAGEMENT

For every adopted open-source component record:

- repository
- version/commit
- license
- modifications
- notices required

Do not copy incompatible code.

Preserve third-party notices.

Pay special attention to Termux-derived code, terminal-emulation libraries, package recipes, X/VNC implementations, Chromium automation and MCP projects.

74. ERROR HANDLING

Failures are first-class product states.

Every integration should expose:

- human-readable error
- technical detail when useful
- retry
- configuration action
- logs/diagnostics

Avoid endless spinners and silent failure.

75. IMPLEMENTATION ORDER

Do not attempt every feature simultaneously.

Phase 1 — Stabilize repository

- audit
- build APK
- fix compile blockers
- inventory mocks

Phase 2 — Design system + auth

- shared theme
- typography
- navigation
- auth
- onboarding

Phase 3 — Files/editor

- real filesystem
- editor
- project import/export

Phase 4 — Ocean Terminal

- PTY
- renderer
- bootstrap
- app-owned prefix/home
- package manager
- package repo integration
- sessions
- process registry

Phase 5 — Preview Router

- local port detection
- web preview
- static preview
- GUI display backend
- APK launch/mirror flow

Phase 6 — Agent runtime

- Koog
- tool registry
- activity events
- permissions
- checkpoints

Phase 7 — Providers/gateway

- secure credentials
- provider registry
- streaming
- model selector
- fallback routing

Phase 8 — MCP/extensions

- transports

- lifecycle
- catalog
- extension permissions

Phase 9 — Browser/research

- browser
- page tools
- research agent

Phase 10 — Device assistance

- MediaProjection
- Accessibility
- OCR
- intents

Phase 11 — Local models

- runtime
- catalog
- downloads
- inference

Phase 12 — Playground

- build tools incrementally using shared infrastructure

Phase 13 — Hardening

- tests
- security
- performance
- lifecycle
- crash handling
- accessibility
- documentation

76. CODING BEHAVIOR FOR CODEX

When modifying the repository:

- inspect before editing
- reuse working architecture
- make cohesive changes
- run build after meaningful batches
- run relevant tests
- investigate compile/runtime errors yourself

- fix root cause
- avoid huge speculative rewrites
- preserve unrelated functionality
- document decisions
- keep the APK buildable

Do not stop at theoretical architecture when repository access and implementation are possible.

77. GIT

Use meaningful commits, for example:

- `feat: implement Ocean Terminal PTY core`
- `feat: add Ocean package repository client`
- `feat: add hardware-local preview router`
- `feat: add provider registry and secure credentials`
- `fix: remove simulated model connection states`

Do not commit:

- API keys
- OAuth tokens
- generated secrets
- local model weights
- large caches
- build artifacts unless intentionally required by the repository

Before pushing:

- build
- test
- inspect diff
- scan for credentials

78. DEFINITION OF DONE

OceanStudio is not done because buttons are visible.

A subsystem is complete only when:

UI:

- reflects real state

Ocean Terminal:

- executes real processes and packages

Provider:

- communicates with a real endpoint

Agent:

- invokes real registered tools

Preview:

- displays a real local/authorized target

Files:

- represent real files

MCP:

- invokes a real server

Extension:

- executes a real installed extension

Browser:

- loads real content

News:

- retrieves real current sources

Local model:

- performs real inference

Device tool:

- uses a user-authorized Android capability

If a feature cannot meet that definition, keep it explicitly incomplete rather than fabricating functionality.

79. PRODUCT PHILOSOPHY

OceanStudio is not a collection of AI-looking cards.

It is an operating environment connecting:

USER
↓
OCEANSTUDIO WORKSPACE
↓
ORCHESTRATOR
↓
TOOLS / AGENTS / PROVIDERS
↓
OCEAN TERMINAL / FILES / BROWSER / DEVICE / SERVICES
↓
REAL OBSERVED RESULT
↓
AGENT VALIDATION
↓
USER

If a process is running, show the real process.

If a file changed, show the real diff.

If a provider answered, show the real provider output.

If an MCP executed, show the real tool result.

If a page was researched, retain source metadata.

If a model is downloading, show actual byte progress.

If an agent is working, show real tool activity and elapsed time rather than simulated thinking.

Build underlying capability first. Let UI expose only what genuinely exists.

80. START NOW

Begin by opening the OceanStudio repository and performing the repository audit.

Then:

1. build the current APK;
2. identify all mocks/placeholders/broken workflows;
3. map the existing architecture;
4. compare the existing terminal to the Ocean Terminal specification;
5. compare preview behavior to the Hardware-Local Preview Router specification;

6. create an implementation plan mapped to the phases above;
7. begin fixing the highest-leverage foundational issues immediately;
8. keep the app buildable while proceeding;
9. validate every subsystem through real tests;
10. maintain concise notes describing what changed, what was verified and what remains.

Do not answer with only a plan if you have repository access and can implement.

Do not claim success from visual inspection alone.

END OF MASTER PROMPT